Análisis técnico preliminar y contraste de observaciones

1. Dictamen y alcance

Proyecto: Asistente de Titulación y Normativa Académica, Grupo 1, UNIANDES. Fecha: 7 de noviembre de 2026, Ecuador (UTC−5). Código revisado: main 0dab82dd9258e9794684ea806b055d61a04d5bb5. Este informe describe implementación y evidencia; no certifica seguridad absoluta ni validez jurídica de la orientación.

La ficha aportada declara una revisión del 6 de noviembre de 2026 a las 18:00 y el commit f8e0a1b. Es una fotografía histórica, no un monitor del servicio. Sus porcentajes 80 % y 64 % no se recalculan automáticamente con un despliegue. No se tiene acceso al código que publica esa ficha para actualizarla.

Conclusión: varias observaciones técnicas ya fueron corregidas y tienen evidencia posterior. El anuncio rojo está parcialmente desactualizado respecto a HEAD y al servicio probado; no es correcto desmentir todo su contenido porque las credenciales legacy siguen recuperables del historial. No hay evidencia suficiente para declarar cumplimiento total de persistencia, revisión humana del corpus, revocación completa y evaluación estricta 6/6 del agente.

2. Evidencia actual frente al anuncio rojo

La ejecución de GitHub Actions 42378740379, intento 2, terminó success para el SHA 0dab82d. Última actualización registrada por GitHub: 7 de noviembre, 09:38:07 de Ecuador (14:38:07 UTC). Backend, frontend, navegador, Gitleaks, contenedor y despliegue figuran aprobados. El job evaluacion-real figura skipped: no debe presentarse como aprobado.

El paso de contratos públicos ejecuta --incluir-modelo y devuelve error si falla cualquier caso seleccionado. El conjunto contiene 35 casos: 32 sin generación y tres generativos (dos directos y uno por proxy SSE). Su conclusión success, junto con el código y el resultado observado durante el seguimiento, acredita la aprobación 35/35 del conjunto. La API de Actions se consultó nuevamente para este informe. No se descargó el JSON del segundo intento por el bloqueo del dominio de artefactos; se conserva una evidencia resumida, no un reporte por caso reconstruido.

El archivo de 32/35 aportado antes por el equipo corresponde al intento fallido anterior. Ese resultado no se borra ni se convierte en aprobado: documenta los tres 503 que motivaron corregir las credenciales de ejecución.

Afirmación de la ficha	Contraste actual	Conclusión defendible
seguridad.py sigue en HEAD	El objeto HEAD:backend/app/seguridad.py no existe en main 0dab82d.	Corregido en código actual.
Hay clave JWT y contraseña fija en el historial	El detector AST histórico devuelve cuatro posiciones en dos commits antiguos, sin imprimir valores.	Sigue pendiente si el requisito exige historial sin secretos.
Cloud Run podría ejecutar el código antiguo	El smoke exige revision SHA exacta, readiness, rutas antiguas 404 y rechazo SSE. Actions intento 2 lo aprobó.	Corregido para la URL y revisión comprobadas. No cubre todas las URLs etiquetadas o copias.
La credencial histórica puede autenticarse en la ruta antigua	Las pruebas actuales exigen 404 en POST /admin/login y POST /admin/recargar-reglamentos.	Las rutas probadas están retiradas; no se volvió a usar la credencial histórica.
No hay pipeline con needs ni despliegue	Workflow con cinco prerrequisitos y promoción de la imagen probada; deploy aprobado.	Observación desactualizada para el commit actual.
No hay 15+ pruebas HTTP ni documentación de semana 4	Existen 35 casos, docs/08-pruebas.md, docs/13-despliegue.md y evidencias.	Corregido, con límites explícitos de cada prueba.

Justificación propuesta para responder al docente

“La observación fue válida para la revisión f8e0a1b del 6 de noviembre. En main 0dab82d el módulo legacy está eliminado y la ejecución 42378740379, intento 2, aprobó el despliegue y los 35 casos públicos, incluida la retirada de las rutas antiguas y generación citada directa y por proxy. Solicitamos actualizar esos puntos de la ficha. Reconocemos que las credenciales persisten en commits antiguos: eliminación del archivo, revocación de valores y limpieza del historial son controles distintos; no declaramos cerrados los dos últimos sin evidencia adicional.”

Qué significa que un secreto esté expuesto

Una contraseña o clave fija almacenada en un commit continúa siendo legible para quien acceda a ese historial aunque el archivo ya no aparezca en la rama actual. El detector histórico señala los commits 1348b065feaba46b6dfea20bf26b1f7d8b455dc8 y 3e38895fff1e60d983b7bf43206e86d69c1be2c2, ruta backend/app/seguridad.py. Este documento solo registra ubicaciones, nunca valores.

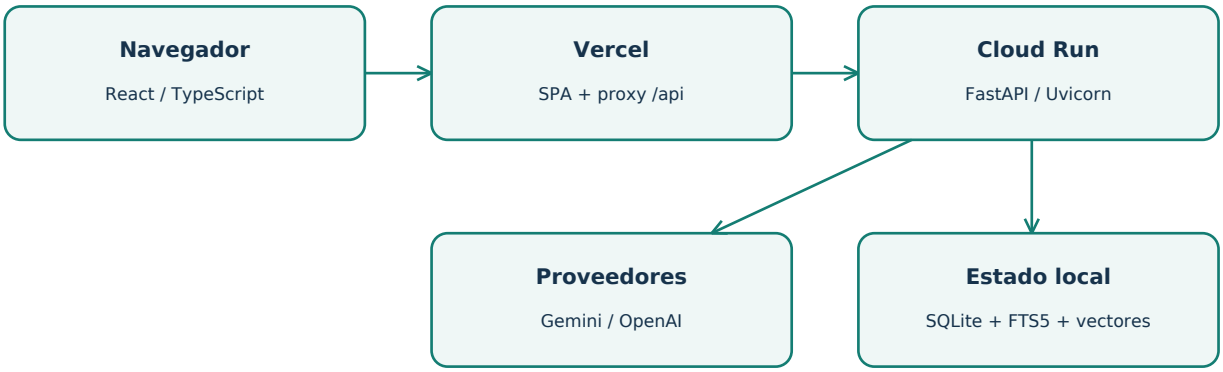
La autenticación actual usa ADMIN_PASSWORD configurada en el servidor y sesiones opacas; no utiliza la clave JWT legacy. Retirar las rutas elimina ese mecanismo en la revisión probada. Eso no demuestra que el valor antiguo haya sido revocado en otros servicios, revisiones etiquetadas o clones. Para declarar “sin secretos en historial” haría falta una limpieza coordinada de referencias y copias, después de revocar cualquier valor reutilizado; no se ha autorizado ni ejecutado una reescritura forzada.

Secret Manager permite almacenar credenciales fuera del repositorio. Tener allí un secreto no equivale a exponerlo públicamente. Las capturas del equipo muestran referencias de GEMINI_API_KEY, OPENAI_API_KEY y ADMIN_PASSWORD; no se exportan sus valores a este informe. WIF evita descargar una clave JSON permanente para desplegar.

3. Matriz de observaciones completas y pendientes

Observación	Evidencia y estado	Qué falta
Tests debilitados	Estados exactos, citas, campos/opciones, token y done, trazas y bloqueo de proveedores en pytest; 104 pruebas locales y CI aprobado.	Mantener protección de PR y no relajar criterios.
Evaluador restaurado	Usa recuperación real y umbrales de config; historial BM25 14/16, híbrido 16/16.	No confundir historial con una nueva corrida híbrida.
Chat público 500	Tres 503 anteriores; tras configurar credenciales, los tres casos generativos pasaron.	Supervisar disponibilidad y cuotas; no garantiza ausencia futura de errores.
Frontend, proxy y despliegue	Vercel SPA y proxy /api; pruebas públicas e imagen de SHA esperado aprobadas.	Cerrar copias antiguas accesibles y conservar evidencia de nuevas entregas.
Pruebas y CI	Tests backend/frontend/navegador y contenedor; Gitleaks y detector activo; contratos públicos obligatorios.	La evaluación real completa del agente quedó omitida en esta ejecución.
Documentación y presentación	Guías de RAG, pruebas, despliegue, decisiones, PDF de entrega y este informe técnico.	El PDF previo conserva un estado histórico; usar este informe fechado como complemento.
Límite de consultas	30 POST por 60 s y proceso para chat, stream y agente; 429 con Retry-After.	Cuota distribuida/por usuario si se escala; el presupuesto es global.
Credenciales	Ausentes como constantes activas detectadas; Secret Manager y WIF usados en despliegue.	Acreditar rotación/revocación de valores históricos y, si se exige, limpieza del historial.
Corpus institucional	PDF + OCR con hashes y 86 fragmentos; approved=false.	Revisión humana de OCR, vigencia y aplicabilidad institucional.
Agente generativo	Evidencia histórica: eligió herramienta 6/6, produjo lista final citada 5/6.	Obtener 6/6 bajo el mismo criterio después de resolver la falta de normativa.
Persistencia	SQLite local y semilla permiten arrancar; max-instances=1 no aporta durabilidad.	Diseñar y probar almacenamiento duradero, respaldo y recuperación.
Uso de IA individual	README reconoce asistencia; no se inventa autoría.	Cada integrante debe declarar y explicar su participación real.

4. Arquitectura y tecnologías



El navegador ejecuta React 19 y TypeScript, compilados con Vite. Vercel sirve los recursos estáticos y aplica fallback de rutas a index.html; /admin y /metricas son rutas de la interfaz, no APIs legacy. La configuración de producción define la URL de Cloud Run y el rewrite /api actúa como proxy, fundamental para mantener la administración en el mismo origen del navegador.

FastAPI valida contratos Pydantic y Uvicorn atiende HTTP en Cloud Run. El contenedor Python 3.12 usa usuario sin privilegios, PORT configurable y semilla léxica sin credenciales de modelos durante el build. Las dependencias declaradas usan rangos; no debe afirmarse que la imagen esté completamente fijada por digest o que use el lockfile de Python: el Dockerfile instala requirements.txt.

SQLite guarda registros JSON de configuración, documentos, prompts, experimentos y acciones; también fragmentos, FTS5 y vectores serializados. No se usa Chroma, una base vectorial externa ni ANN. FTS5/BM25 aporta búsqueda léxica; el modo híbrido calcula similitud coseno en Python y fusiona rankings con RRF. Es una arquitectura local acotada a 5000 fragmentos.

Los SDK google-genai y openai son los clientes de proveedores. pypdf extrae PDF; defusedxml participa en la lectura segura de documentos XML; python-multipart procesa cargas. pytest/pytest-cov, Ruff, Vitest, Playwright, pip-audit y Gitleaks participan en la calidad. reportlab y PyMuPDF generan y revisan este PDF, no forman parte del flujo de consulta del usuario.

5. Ingesta, conocimiento e indexación

scripts/ingest.py lee manifiesto y config/fuentes.yaml. Comprueba hashes SHA-256 de originales y OCR antes de modificar documentos. Los metadatos incluyen título, tipo, URL, fechas, aprobación y habilitación. No comprueba por sí mismo vigencia jurídica real ni autoriza una fuente: eso lo decide el equipo.

La extracción conserva páginas y secciones. El fraccionamiento toma chunk_size 1400 y overlap 160 como valores iniciales, con controles de rango y solapamiento. Los IDs y revisiones permiten conectar respuesta, fragmento y original. Una ingesta repetida reutiliza documentos y vectores idénticos cuando la firma del índice coincide.

Una fuente se puede recuperar solo si está aprobada, habilitada, vigente y tiene un índice de la revisión/configuración adecuada. Reemplazar un archivo fuerza approved=false para una nueva revisión humana. El corpus conocido contiene 961 fragmentos de fuentes aprobadas y 86 del instructivo UNIANDES pendiente; no se suman los 86 a la evidencia elegible.

En modo lexical, indexar no llama al proveedor. En hybrid genera embeddings de 768 dimensiones por lotes de hasta 16, con caché por texto, modelo y tarea. Gemini usa gemini-embedding-001; OpenAI permite text-embedding-3-small. El código limita el presupuesto de textos y reintentos de embeddings. La escritura del índice usa una transacción; un fallo del proveedor conserva el índice anterior.

6. Secuencia de una consulta y llamadas a modelos



1. La interfaz reúne pregunta, contexto académico y modelo preferido. ConsultaChat exige pregunta de 3 a 2000 caracteres y limita cada campo de contexto. Los únicos modelos preferidos admitidos son gemini y openai.

2. El middleware consume el presupuesto de consultas antes de validar el cuerpo. Los alias /api/chat, /api/chat/stream y /api/agente comparten el mismo límite por proceso. Una entrada inválida también consume presupuesto; GET health y OPTIONS no lo consumen.

3. El flujo bloquea marcadores de inyección antes de recuperar evidencia. Son reglas heurísticas, no una prueba de resistencia a toda forma de inyección. El contexto y los documentos se tratan como datos.

4. RAG filtra fuentes elegibles, extrae términos, busca hasta 40 candidatos FTS5 y aplica cobertura léxica mínima. En modo híbrido y con vectores compatibles genera un embedding de consulta, calcula coseno, aplica umbral y fusiona posiciones mediante RRF k=60. top_k inicial es 4.

5. Si no hay evidencia suficiente, devuelve abstención local sin generación. Una consulta fuera de dominio puede rechazarse localmente. Cambios de consulta de checklist o borrador de tutor se preparan para buscar la evidencia pertinente.

6. Con evidencia, se carga el prompt activo y reglas del flujo. El agente envía consulta, contexto, evidencia, herramientas disponibles y resultados previos al modelo. El modelo puede devolver respuesta final JSON o pedir una sola herramienta por vuelta.

7. El backend valida nombre y argumentos de la herramienta, ejecuta la función Python y devuelve el resultado al siguiente turno. El tope es cuatro vueltas; no es un agente de navegación libre ni ejecuta código arbitrario generado.

8. La salida final pasa BorradorRAG: estado, texto, IDs citados y campos de aclaración. Un ID inventado, falta de citas en una respuesta normativa, revisión que cambió o JSON inválido producen abstención segura. Las citas se resuelven a título, sección, página, revisión, hash y URL.

9. JSON se devuelve directamente o se transforma en eventos SSE token y done. En esta implementación primero se completa generar_respuesta y después se divide el texto por espacios: no es streaming de tokens directamente desde Gemini/OpenAI. El evento done contiene el contrato final y las fuentes.

10. La telemetría técnica registra modelo, tiempos, tokens reportados, estado y categoría de fallo. No escribe preguntas, respuestas ni claves en el archivo público de métricas. Las trazas y experimentos administrativos sí pueden contener textos de casos y resultados, con acceso administrativo.

Cuándo se usa cada modelo

Situación	Generación	Embeddings / otras llamadas
health, ready, calendario, sesiones y configuración	No	No. ready inspecciona corpus/índice local.
Rechazo local de inyección	No	No; ocurre antes de RAG.
Consulta sin evidencia en lexical	No	No; BM25 y abstención local.
RAG híbrido con vectores compatibles	Solo si luego hay evidencia	Embedding de la consulta; si falla, se conserva BM25.
Ingesta o indexación híbrida	No genera respuestas	Embeddings de documentos, con caché.
Chat/agente con evidencia	Preferido por el usuario; por defecto Gemini	Puede hacer varias llamadas, hasta el límite del agente.
Comparación administrativa	Solo el proveedor seleccionado; sin fallback	Recuperación compartida por caso; no hay un juez IA adicional.

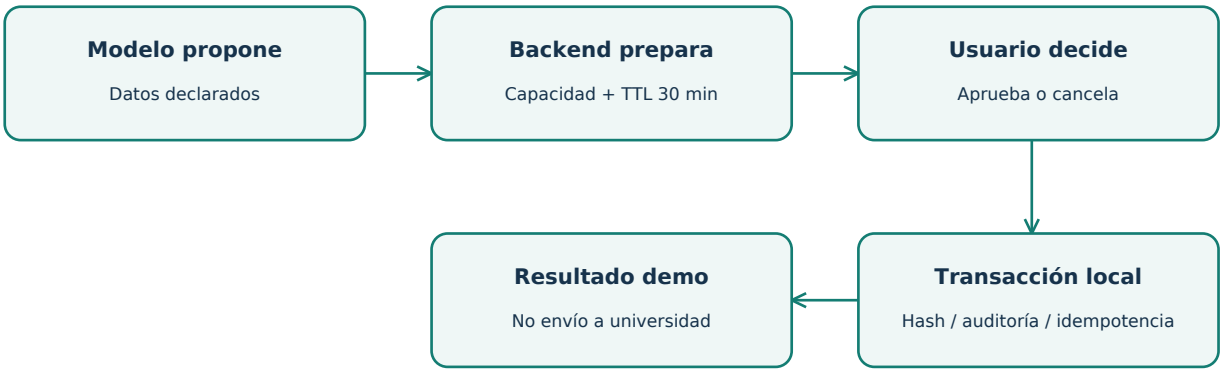
Settings declara Gemini gemini-3.5-flash-lite y respaldo OpenAI gpt-4.1-mini; las variables de ejecución pueden cambiar los nombres. GEMINI_API_KEY y OPENAI_API_KEY habilitan sus clientes. AI_API_KEY aparece en el ejemplo de entorno pero no es leída por Settings: no sustituye a ninguna de esas claves.

El orden real lo decide modelo_preferido y la disponibilidad de claves: primero el preferido y después el otro. Si el principal falla se intenta el respaldo configurado. Si todos fallan o no hay proveedores habilitados, el chat devuelve 503. El tiempo de espera generativo inicial es 15 s por intento y LLM_RETRIES=0; un flujo con varias vueltas y dos proveedores puede durar más de 15 s en total. Las simulaciones administrativas deshabilitan fallback para comparar el proveedor elegido.

La aprobación 35/35 acredita tres respuestas generativas correctas bajo sus contratos, no que todas provinieran de Gemini. Para atribuir proveedor hay que consultar modelo_utilizado y fallback_activado o la telemetría correspondiente. Este informe no inventa esa atribución.

7. Herramientas y confirmación humana

Herramienta del agente	Datos principales	Ejecución y límites
buscar_fragmentos	pregunta	Recupera pasajes con RAG; no redacta por sí sola.
consultar_requisitos	modalidad	Busca evidencia para interpretar requisitos; no certifica una lista oficial completa.
generar_checklist	perfil_estudiante	Interpreta nivel/modalidad declarados, filtra evidencia incompatible y devuelve items con citas y estado pendiente_de_verificar.
calcular_plazo	fecha_inicio ISO, dias_habiles	Python determinista; máximo 730 días, excluye fines de semana y fecha inicial; no incluye feriados ni fija plazos institucionales.
registrar_solicitud_tutor	carrera, modalidad, tema	Solo propone borrador demo si el usuario lo pidió y aportó los datos. Requiere confirmación posterior.



La lista PERMITIDAS es cerrada. Los argumentos se validan con Pydantic y no se confía en el nombre de herramienta elegido por el modelo. El calendario requiere fecha y cantidad explícitas en la pregunta. El checklist no puede inventar un perfil distinto del declarado. Antes de usar evidencia se verifica que siga vigente y actual.

La propuesta de tutor genera una capacidad aleatoria y guarda solo su hash junto con la acción pendiente; TTL inicial de 30 minutos. El usuario ve datos y alcance. DecisionIn exige token y aprobar booleano estricto. La decisión se ejecuta en transacción; repetir la misma decisión no duplica registros y una decisión contradictoria devuelve conflicto. No se envía nada a la universidad, no se asigna tutor y el actor demo no es una identidad universitaria autenticada.

8. Administración y seguridad

/api/admin usa sesión opaca aleatoria, cookie HttpOnly, SameSite=Strict y Secure en producción. El servidor conserva el hash del token y expiración en memoria. La contraseña procede de ADMIN_PASSWORD; las comparaciones usan hmac.compare_digest sobre digest. No se usa el JWT histórico en la administración actual.

Las mutaciones administrativas requieren X-Admin-Request: 1 y validación del origen cuando se envía Origin. No se confía en X-Forwarded-Host para autorizar; solo se aceptan mismo host u orígenes configurados. El login limita a cinco fallos por IP durante cinco minutos. Esta defensa depende de la dirección que vea el servidor/proxy y no sustituye un control de plataforma distribuido.

CORS permite el origen explícito de Vercel y no cookies cruzadas; las cookies administrativas circulan por el proxy del mismo origen. Las respuestas administrativas tienen Cache-Control: no-store y X-Content-Type-Options: nosniff. El middleware limita tamaño declarado de cargas y la carga de archivo lee como máximo el límite admitido más un byte (10 MiB para el archivo). La ruta de descarga valida que el archivo resuelto siga dentro del directorio de datos.

El sistema controla revisiones optimistas para no sobrescribir documentos, prompts o configuración modificados por otro actor. Las fechas y aprobaciones siguen requiriendo criterio humano. Los controles de aplicación no equivalen a una auditoría exhaustiva de IAM, red, supply chain o pruebas de penetración.

9. Catálogo completo de endpoints

El anexo siguiente se genera desde app.openapi() del commit revisado y se complementa con lectura de los handlers. Incluye las 31 operaciones de negocio de FastAPI, sus entradas, autorización, salidas y posibles llamadas a modelos. Las descripciones de errores reflejan controles del código, no solo el esquema automático 422.

9.1. POST /api/admin/login

Acceso y función. Login: username y password; mutación con cabecera X-Admin-Request y origen admitido.

Cuerpo (application/json). username: string (obligatorio); minLength=1, maxLength=80; password: string (obligatorio); minLength=1, maxLength=200

Salida y efectos. 200 username y cookie de sesión; 401 credencial inválida, 403 guard, 429 cinco fallos, 503 falta configuración. Sin LLM.

9.2. GET /api/admin/session

Acceso y función. Cookie de sesión administrativa vigente.

Salida y efectos. 200 username; 401 sin sesión. Sin LLM.

9.3. POST /api/admin/logout

Acceso y función. Cookie vigente, guard de mutación; sin cuerpo.

Salida y efectos. 200 ok; elimina sesión y cookie. Sin LLM.

9.4. GET /api/admin/bootstrap

Acceso y función. Cookie vigente; sin cuerpo.

Salida y efectos. 200 prompts, documentos públicos, configuración, umbrales, casos y modelos configurados (booleanos, nunca claves). Sin LLM.

9.5. POST /api/admin/prompts

Acceso y función. PromptInput: content y description; cookie y guard.

Cuerpo (application/json). content: string (obligatorio); minLength=20, maxLength=40000; description: string (obligatorio); minLength=3, maxLength=300; expected_revision: opcional/unión

Salida y efectos. 200 prompt con nueva versión (hasta 999) y revisión, escribe archivo y auditoría. 409 límite. Sin LLM.

9.6. POST /api/admin/prompts/{version}

Acceso y función. Versión en ruta; PromptInput con expected_revision; cookie y guard.

Cuerpo (application/json). content: string (obligatorio); minLength=20, maxLength=40000; description: string (obligatorio); minLength=3, maxLength=300; expected_revision: opcional/unión

Parámetros. version (path, obligatorio)

Salida y efectos. 200 revisión incrementada; 404 versión ausente, 409 conflicto de revisión. Sin LLM.

9.7. POST /api/admin/config

Acceso y función. RAGConfig con prompt activo, chunk_size, overlap, top_k, estrategia, embedding_model, umbrales y revision; cookie y guard.

Cuerpo (application/json). active_prompt: integer (obligatorio); minimum=1.0, maximum=999.0; chunk_size: integer; minimum=300.0, maximum=4000.0; overlap: integer; minimum=0.0, maximum=1000.0; top_k: integer; minimum=1.0, maximum=10.0; strategy: string; enum=['lexical', 'hybrid']; embedding_model: string; enum=['gemini-embedding-001', 'text-embedding-3-small']; min_similarity: number; minimum=0.0, maximum=1.0; min_lexical_coverage: number; minimum=0.0, maximum=1.0; revision: integer (obligatorio); minimum=1.0

Salida y efectos. 200 configuración revisada; 409 conflicto, 422 prompt inexistente o parámetros inválidos. No reindexa ni llama a un proveedor en esta operación.

9.8. POST /api/admin/documents

Acceso y función. Multipart: metadata JSON DocMeta y file; cookie y guard.

Cuerpo (multipart/form-data). metadata: string (obligatorio); file: string (obligatorio)

Salida y efectos. 200 documento guardado con revisión; 422 archivo/formato/metadatos inválidos, 409 máximo 100 documentos, 413 exceso declarado. No genera respuestas ni indexa automáticamente.

9.9. POST /api/admin/documents/{doc_id}

Acceso y función. doc_id y DocUpdate con expected_revision; cookie y guard.

Cuerpo (application/json). title: string (obligatorio); minLength=3, maxLength=200; kind: string; enum=['normativa', 'informativo']; source_url: string; maxLength=1000; valid_from: string (obligatorio); valid_to: opcional/unión; approved: boolean; enabled: boolean; notes: string; maxLength=2000; expected_revision: integer (obligatorio); minimum=1.0

Parámetros. doc_id (path, obligatorio)

Salida y efectos. 200 metadatos actualizados; 404 no encontrado, 409 revisión concurrente. Sin LLM.

9.10. POST /api/admin/documents/{doc_id}/replace

Acceso y función. doc_id; multipart metadata, expected_revision, file; cookie y guard.

Cuerpo (multipart/form-data). metadata: string (obligatorio); expected_revision: integer (obligatorio); file: string (obligatorio)

Parámetros. doc_id (path, obligatorio)

Salida y efectos. 200 nueva revisión con approved=false; 404 ausente, 409 conflicto, 422 archivo inválido. Exige nueva aprobación humana; no genera texto.

9.11. GET /api/admin/documents/{doc_id}/file

Acceso y función. doc_id; query revision opcional; cookie vigente.

Parámetros. doc_id (path, obligatorio); revision (query, opcional)

Salida y efectos. 200 archivo de revisión actual o histórica; 404 si no existe o ruta fuera del directorio. No ejecuta modelos.

9.12. GET /api/admin/history

Acceso y función. Query resource obligatorio; cookie vigente.

Parámetros. resource (query, obligatorio)

Salida y efectos. 200 hasta 100 entradas de auditoría before/after. Las páginas internas se filtran en documentos. Sin LLM.

9.13. GET /api/admin/rag/chunks

Acceso y función. Query doc_id obligatorio; cookie vigente.

Parámetros. doc_id (query, obligatorio)

Salida y efectos. 200 hasta 512 fragmentos con ID, revisión, título, sección, página y texto. Sin LLM.

9.14. POST /api/admin/rag/index/{doc_id}

Acceso y función. doc_id; cookie y guard.

Parámetros. doc_id (path, obligatorio)

Salida y efectos. 200 índice/fragmentos y traza; 404 ausente, 409 conflicto, 422 límite/entrada, 503 fallo de embeddings. Solo llama a embeddings si estrategia híbrida; conserva índice previo ante error.

9.15. POST /api/admin/rag/search

Acceso y función. Search.query; cookie y guard.

Cuerpo (application/json). query: string (obligatorio); minLength=3, maxLength=2000

Salida y efectos. 200 chunks, estrategia efectiva, warning y trace. Puede llamar embeddings en modo híbrido; no genera respuesta de chat.

9.16. GET /api/admin/analysis

Acceso y función. GET: cookie vigente. POST: AnalysisInput con 1–5 versiones, modelo, 1–10 casos, contexto y use_rag; cookie y guard.

Salida y efectos. GET 200 lista resumida de hasta 30 trabajos. POST 202 id/status; 409 trabajo ocupado, 422 versión inválida. En segundo plano genera con proveedor elegido, sin fallback ni escrituras demo; no hay juez IA.

9.17. POST /api/admin/analysis

Acceso y función. GET: cookie vigente. POST: AnalysisInput con 1–5 versiones, modelo, 1–10 casos, contexto y use_rag; cookie y guard.

Cuerpo (application/json). versions: array (obligatorio); minItems=1, maxItems=5; model: string; enum=['gemini', 'openai']; cases: array (obligatorio); minItems=1, maxItems=10; use_rag: boolean

Salida y efectos. GET 200 lista resumida de hasta 30 trabajos. POST 202 id/status; 409 trabajo ocupado, 422 versión inválida. En segundo plano genera con proveedor elegido, sin fallback ni escrituras demo; no hay juez IA.

9.18. GET /api/admin/analysis/{job_id}

Acceso y función. job_id y cookie vigente.

Parámetros. job_id (path, obligatorio)

Salida y efectos. 200 experimento, resultados, evidencia y traza; 404 ausente. La lectura no llama a modelos.

9.19. GET /api/herramientas

Acceso y función. Público; sin cuerpo.

Salida y efectos. 200 esquemas de cinco herramientas, salidas, max_iteraciones y requieren_ok. No invoca LLM ni ejecuta herramienta.

9.20. POST /api/herramientas/calcular-plazo

Acceso y función. PlazoIn: fecha_inicio ISO y dias_habiles entero estricto 0–730; público.

Cuerpo (application/json). fecha_inicio: string (obligatorio); dias_habiles: integer (obligatorio); minimum=0.0, maximum=730.0

Salida y efectos. 200 fecha_resultado y alcance de simulación; 422 entrada inválida. Determinista, sin LLM ni feriados.

9.21. POST /api/herramientas/consultar-requisitos

Acceso y función. RequisitosIn.modalidad; público.

Cuerpo (application/json). modalidad: string (obligatorio); minLength=3, maxLength=120

Salida y efectos. 200 ConsultaOut con pasajes citables y alcance. Recuperación local/híbrida; puede usar embeddings pero no generación.

9.22. POST /api/herramientas/generar-checklist

Acceso y función. ChecklistIn.perfil_estudiante; público.

Cuerpo (application/json). perfil_estudiante: string (obligatorio); minLength=3, maxLength=500

Salida y efectos. 200 ChecklistOut con perfil, items citados y completo=false. Filtros deterministas y RAG; puede usar embeddings pero no generación.

9.23. POST /api/herramientas/agente/confirmar/{action_id}

Acceso y función. action_id y DecisionIn token/aprobar; capacidad de confirmación, no sesión admin.

Cuerpo (application/json). token: string (obligatorio); minLength=32, maxLength=100; aprobar: boolean (obligatorio)

Parámetros. action_id (path, obligatorio)

Salida y efectos. 200 DecisionOut; 404 ID/token inválido, 410 vencida, 409 decisión contradictoria. Alias de confirmación; guarda o cancela solo borrador local, sin LLM.

9.24. POST /api/agente/confirmar/{action_id}

Acceso y función. action_id y DecisionIn token/aprobar; capacidad de confirmación, no sesión admin.

Cuerpo (application/json). token: string (obligatorio); minLength=32, maxLength=100; aprobar: boolean (obligatorio)

Parámetros. action_id (path, obligatorio)

Salida y efectos. 200 DecisionOut; 404 ID/token inválido, 410 vencida, 409 decisión contradictoria. Idempotente y auditado. Sin LLM.

9.25. GET /health

Acceso y función. Público; sin cuerpo.

Salida y efectos. 200 status, service, version y revision. Comprueba proceso e identidad declarada; no corpus ni proveedor.

9.26. GET /ready

Acceso y función. Público; sin cuerpo.

Salida y efectos. 200 status ready, fragmentos_elegibles, estrategia, proveedor_verificado=false; 503 sin corpus/índice elegible. No llama a modelos.

9.27. GET /api/sources/{doc_id}/{revision}

Acceso y función. doc_id y revision; público.

Parámetros. doc_id (path, obligatorio); revision (path, obligatorio)

Salida y efectos. 200 archivo solo si la fuente está vigente/aprobada y la revisión coincide; 404 en otro caso. No genera texto.

9.28. GET /api/metricas

Acceso y función. Público; sin cuerpo.

Salida y efectos. 200 agregados y últimos 50 eventos técnicos de una ventana de hasta 2000; sin prompts, respuestas, claves ni datos personales en esos eventos. No llama a modelos.

9.29. POST /api/chat

Acceso y función. ConsultaChat: pregunta, modelo_preferido y contexto académico opcional; público con presupuesto global.

Cuerpo (application/json). pregunta: string (obligatorio); minLength=3, maxLength=2000; carrera: opcional/unión; modalidad: opcional/unión; modelo_preferido: string; enum=['gemini', 'openai']; curso: opcional/unión; periodo: opcional/unión; jornada: opcional/unión; tema: opcional/unión; objetivo: opcional/unión; sede: opcional/unión; modalidad_titulacion: opcional/unión

Salida y efectos. 200 RespuestaAsistente: respuesta/abstencion/aclaracion/rechazo/confirmacion; citas, traza, campos, modelo, fallback y tiempo. 422 validación, 429 presupuesto, 503 proveedor. Modelos solo según flujo descrito.

9.30. POST /api/agente

Acceso y función. Mismo ConsultaChat y presupuesto que /api/chat.

Cuerpo (application/json). pregunta: string (obligatorio); minLength=3, maxLength=2000; carrera: opcional/unión; modalidad: opcional/unión; modelo_preferido: string; enum=['gemini', 'openai']; curso: opcional/unión; periodo: opcional/unión; jornada: opcional/unión; tema: opcional/unión; objetivo: opcional/unión; sede: opcional/unión; modalidad_titulacion: opcional/unión

Salida y efectos. 200 mismo contrato/orquestador de chat; no es un segundo motor distinto. Mismos errores y reglas de generación.

9.31. POST /api/chat/stream

Acceso y función. ConsultaChat; público, comparte presupuesto. El flujo se resuelve antes de iniciar SSE.

Cuerpo (application/json). pregunta: string (obligatorio); minLength=3, maxLength=2000; carrera: opcional/unión; modalidad: opcional/unión; modelo_preferido: string; enum=['gemini', 'openai']; curso: opcional/unión; periodo: opcional/unión; jornada: opcional/unión; tema: opcional/unión; objetivo: opcional/unión; sede: opcional/unión; modalidad_titulacion: opcional/unión

Salida y efectos. 200 text/event-stream con token y done del contrato completo; errores previos 422/429/503 como JSON. Mismo agente, evidencia y fallback que chat.

Las rutas automáticas GET /openapi.json, /docs, /docs/oauth2-redirect y /redoc sirven contrato y documentación; no ejecutan modelos por sí mismas. Las rutas /, /admin y /metricas pertenecen a la SPA de Vercel. /admin/login y /admin/recargar-reglamentos son rutas legacy retiradas, no endpoints válidos del backend actual.

10. Pruebas implementadas y qué demuestran

Capa	Implementación	Alcance acreditado
Backend	pytest y pytest-cov; 104 pruebas, cobertura 88,73 %; umbral exigido 85 %.	Contratos, RAG, herramientas, confirmaciones, admin, producción, evaluadores, límites y falsos aprobados del verificador.
Aislamiento de proveedores	conftest desactiva lectura del .env, vacía claves y bloquea métodos reales; estado, prompts y métricas temporales.	Las pruebas no consumen proveedores ni dependen de credenciales personales.
Frontend	Vitest y Testing Library; 18 pruebas históricas conservadas, job actual aprobado; TypeScript y Vite build.	Componentes, contexto y contratos simulados; no valida por sí solo el backend real.
Navegador	Playwright; suite admin de 10 escenarios con API aislada y modelo doble.	Login, prompts, fuentes, análisis, checklist, confirmación, contexto, trazas y adaptación móvil.
Calidad y dependencias	Ruff, detector activo, Gitleaks --all, npm audit y pip-audit.	Hallazgos conocidos dentro de cada alcance; no prueba ausencia universal de secretos.
Contenedor	Build, usuario no root, semilla, smoke de SHA/readiness/herramientas/calendario/legacy/SSE.	Imagen arrancable y contrato local antes de promover el mismo artefacto.
Producción pública	35 casos en eval/casos_http.json; exit 1 ante cualquier fallo; tres generativos.	URLs y revisión concreta, con autenticación administrativa ausente y pruebas no destructivas; no prueba login admin real.
RAG y agente reales	eval/evaluar.py, comparar.py, evaluar_flujo.py, evaluar_agente_real.py.	Criterios y resultados reales separados de dobles. La corrida actual no ejecutó evaluacion-real.

Los 35 casos públicos se distribuyen en cuatro de frontend/proxy, dos de proceso/corpus, dos de retirada legacy, cuatro de administración, uno de catálogo de agente, tres de validación de chat, tres de inyección, dos CORS, seis calendario, cuatro herramientas, uno de fuente inexistente y tres de generación citada. No se acepta 200 como suficiente: se validan estados, revisiones, documento/artículo, fragmentos y evento final SSE.

Los falsos aprobados del verificador se prueban con documentos incorrectos, revisión vieja, 500, SSE truncado, respuestas vacías o sin modelo y citas por proxy sin evento final. La lectura HTTP acotada a 1 MiB evita truncar respuestas válidas por el antiguo límite de 8 KiB. El runner omite el JSON libre de la respuesta al guardar el reporte por casos; no almacena claves ni capacidades de confirmación.

La evaluación histórica del RAG léxico logró 14/16 positivos y 4/4 negativas; la comparación híbrida histórica 16/16. El evaluador comprueba documentos/artículos esperados y abstención, no equivalencia semántica perfecta. La evaluación generativa real de checklist produjo 5/6 finales citadas: no se convierte en 6/6 por aprobar tres consultas HTTP diferentes.

11. Secuencia CI/CD y configuración operativa

Push o PR ejecuta secretos, backend, frontend, navegador y contenedor. desplegar depende del éxito de esos cinco y solo se habilita en main por push o workflow_dispatch. evaluacion-real es manual y requiere activar evaluar_modelo_real; es independiente de los cinco controles y no debe confundirse con los tres casos generativos posdespliegue.

El job contenedor incorpora APP_REVISION=SHA, prueba la imagen y la guarda como artefacto. El job desplegar usa OIDC de GitHub y WIF para suplantar github-deploy, carga el mismo artefacto, lo publica en Artifact Registry y actualiza Cloud Run. No reconstruye después de probar. El workflow conserva variables y referencias de secretos con actualización merge y configura cookie Secure, CORS y revisión.

Las variables GCP_PROJECT_ID, GCP_AR_REPOSITORY, GCP_WORKLOAD_IDENTITY_PROVIDER y GCP_SERVICE_ACCOUNT son identificadores, no claves LLM. La cuenta de despliegue necesita escritura en el repositorio Docker, despliegue sobre el servicio y actAs sobre la cuenta de ejecución. La cuenta titulacion-run necesita secretAccessor en cada secreto utilizado. Los permisos WIF, despliegue y lectura de secretos son controles distintos.

Tras desplegar se comprueba SHA, corpus, herramientas, calendario, SSE y rutas antiguas; después se ejecutan obligatoriamente los 35 casos contra Cloud Run/Vercel. Un fallo deja el job rojo aunque Cloud Run ya haya recibido la revisión; no existe rollback automático. Repetir un job fallido puede desplegar de nuevo, por lo que debe preservarse la configuración válida de ejecución.

Cloud Run usa max-instances=1, mínimo 0 y memoria 1 GiB. No garantiza persistencia ni exclusión de revisiones durante un cambio de tráfico. La sesión y el límite viven en memoria; SQLite y los archivos en el contenedor son efímeros. RAG_DATA_DIR solo selecciona ubicación. No se propone SQLite sobre Cloud Storage FUSE como solución de bloqueo y durabilidad.

12. Criterios para cierre y sustentación

Puede solicitarse actualización de la ficha para: eliminación de seguridad.py en HEAD, backend de SHA conocido, CI/CD con despliegue, pruebas HTTP generativas, rutas legacy retiradas, pruebas estrictas, límite de consultas y documentación actualizada. La prueba se refiere al commit, intento y URL concretos; no se puede generalizar a cualquier copia del proyecto.

No se deben marcar todavía como totalmente cerrados: secretos en historial, rotación de cualquier valor reutilizado, normativa institucional/OCR aprobado, seis listas generativas correctas, almacenamiento duradero y declaración individual de uso de IA. Cada pendiente tiene evidencia de cierre distinta: acta de revisión del corpus, reporte del evaluador estricto, prueba de recuperación tras reinicio y constancia de gestión de secretos sin divulgar valores.

Para sustentación conviene demostrar una respuesta con cita y abrir su fuente; una abstención sin evidencia; un rechazo local de inyección; un checklist con estado pendiente; un calendario con fecha y días explícitos; y una propuesta de tutor cancelada o confirmada con datos ficticios. Explicar qué pasos usaron modelo y cuáles fueron deterministas. No afirmar que una cita estructural garantiza toda la respuesta ni que el sistema emite resoluciones universitarias.

13. Fuentes y reproducibilidad

Fuente primaria de código: repositorio felipet73/SeminarioComplexivoGrupo1, main 0dab82dd9258e9794684ea806b055d61a04d5bb5. Evidencia remota: https://github.com/felipet73/SeminarioComplexivoGrupo1/actions/runs/42378740379/attempts/2 . Capturas aportadas por el equipo: ficha fechada el 6 de noviembre, referencia f8e0a1b.

Archivos clave: backend/app/main.py; config.py; agente.py; services/chat_flow.py; services/rag.py; services/llm.py; services/actions.py; services/analysis.py; admin_routes.py; admin_auth.py; tools/academicas.py; config/parametros.yaml; config/fuentes.yaml; scripts/ingest.py; scripts/check_secrets.py; scripts/pruebas_funcionales.py; frontend/vercel.json; .github/workflows/ci.yml.

Este documento se genera con scripts/generar_analisis_tecnico.py. El inventario se exporta con app.openapi() sin llamadas a proveedores. La evidencia resumida, hashes y resultados locales acompañan el informe. Se preservan los reportes históricos sin sobrescribirlos. El informe es preliminar y requiere revisión del equipo antes de presentarlo al tribunal; no se alteró la ficha externa ni se publicaron cambios de infraestructura para redactarlo.

14. Ampliación forense: falsos positivos y credenciales funcionales

La revisión adicional de .gitleaksignore confirmó seis fingerprints: dos apuntan a auditoria-python.json en los commits 869404be y 8f3369c9, y cuatro a campos Match/Secret de gitleaks-local.json en a6b12a7. Los dos originales son un token de ejemplo del PoC de filelock 4.1.0 (CVE-2026-112934) incluido en el texto del advisory por pip-audit; los otros cuatro son copias del reporte. No son credenciales operativas del proyecto. Las exclusiones de esos seis hallazgos son justificables y no apuntan al módulo legacy.

El caso distinto es seguridad.py: el commit 1348b065 (5/11, 15:25 Ecuador) introdujo una clave literal de firma JWT en línea 10 y una contraseña literal pasada a hash en línea 20. En 3e38895 (5/11, 16:17) permanecen en líneas 9 y 19, con import corregido a app.seguridad. Se usaban en jwt.encode/decode y validación del login; no son un fixture inactivo. La comprobación histórica remota anterior al despliegue registró aceptación de la credencial.

En 9434e2d (6/11, 16:19) se dejó de registrar el mecanismo, conservando el archivo. En 08ef882 (6/11, 17:02) se eliminó. La revisión 0dab82d y URL probadas ya no aceptan ese mecanismo: módulo ausente, rutas antiguas 404 y autenticación actual con sesión opaca. No se afirma riesgo cero en cualquier copia, ni revocación de valores reutilizados: el historial sigue conteniéndolos.

El motivo comprobable de su incorporación fue implementar autenticación JWT de rutas administrativas con valores fijos dentro del código. No se atribuye intención o autoría humana no documentada. Las ubicaciones, fechas, relación entre exclusiones y evidencia están en docs/15-auditoria-credenciales-historicas.md y su JSON, sin valores secretos.

15. Contraste con los dos PDF oficiales de sustentación y entrega

Los PDF aportados por el equipo exigen demo pública de 12 minutos más tres de preguntas, vídeo real de respaldo de dos minutos, intervención de todos los integrantes y las once casillas s4a–s4k. La entrega oficial es lunes 9 de noviembre de 2026 a las 23:59 de Ecuador; prórroga hasta miércoles 11 a las 23:59. No basta con que la definición del pipeline exista o con un test local.

Punto	Estado que puede declararse	Falta para cierre total
s4a URL/health	Cumplido en la revisión probada	Demostración en vivo en navegador privado.
s4b credenciales e historial	Parcial; mecanismo legacy retirado	Historial limpio y constancia de rotación/no reutilización.
s4c índice tras reinicio	Base restaurable en imagen	Ensayo remoto con cita antes/después; durabilidad de cambios si se exige conservarlos.
s4d pruebas públicas	35/35 acreditado por Actions	Adjuntar reporte por caso del intento 2 y su p95; no reciclar latencia del intento fallido.
s4e evaluación comparada	Baseline final local 14/16 y 4/4; flujo doble 20/20	Repetir el conjunto generativo completo y seis checklists bajo el mismo criterio.
s4f tasa/errores	Cumplido en alcance por proceso	No confundir con cuota distribuida.
s4g IA/datos	Avisos educativos y minimización visibles	Mantener límites y revisar tratamiento del proveedor antes de uso institucional.
s4h instalación/decisiones	Documentación actualizada	Ensayo medido de diez minutos por otro integrante.
s4i IA/aportes	Tabla y asistencia declaradas	Declaración individual confirmada.
s4j PDF de sustentación	Presentación de seis páginas actualizada	Ensayar, elegir versión y acreditar uso en vivo.
s4k evidencias/tribunal	Evidencia técnica presente	Capturas, vídeo y observaciones reales del tribunal con correcciones/commits.

La recuperación léxica local se repitió el 7 de noviembre y conserva 14/16 positivos, 4/4 negativas y cero errores; los contratos con doble pasaron 20/20. No se fabricó una nueva evaluación generativa sin credenciales locales. El estado final está centralizado en docs/14-checklist-entrega-final.md y los registros históricos tienen referencia a ese estado sin borrar sus fallos originales.

El ZIP oficial tiene límite de 20 MB y debe excluir .env, venv, node_modules, caches y .git. La medición local del proyecto conocido con ZIP DEFLATE nivel 9 produjo 28.818.950 bytes (27,48 MiB), por encima de 20 MB; el detalle está en docs/evidencias/semana-4/tamano-paquete-2026-11-07.json. Debe resolverse mediante optimización validada de recursos/hashes o acuerdo con el docente para otra modalidad de corpus. No se eliminan originales referenciados por el manifiesto ni se afirma que el paquete esté listo. El ZIP y GitHub deben corresponder a la misma versión final.